TECHNICAL READINESS DOCUMENT

Project: Autonomous AI Video Editor for Insta360 Tracking

Author:	Sourabh Zunke (Project Consultant)	**Date:**	May 2026
Target System:	NucBox_EVO-T1 (Local Workstation)	**Version:**	1.0.0 (Baseline Master)

1. Executive Summary & Objective

The objective of this project is to develop an automated, hardware-accelerated local desktop pipeline that ingests raw and stitched 360-degree action sports footage (skiing, white-water rafting, high-altitude hiking) and utilizes advanced Computer Vision (CV) to perform smooth, cinematic camera reframing.

By leveraging localized Deep Learning tracking frameworks alongside modern predictive mathematical models, the engine mimics a professional human camera operator. It eliminates high-frequency camera vibrations, tracks specific pre-defined or arbitrarily designated sports objects, and outputs standard commercial-ready aspect ratios without relying on external cloud processing infrastructures.

2. System Hardware Baseline

The software deployment and model engineering are custom-tailored to utilize the specific integrated architecture of the local host system, bypassing standard CUDA dependencies through Intel hardware primitives.

Host Machine Model	NucBox_EVO-T1
Processor (CPU)	Intel(R) Core(TM) Ultra 9 285H (2.90 GHz Base, with integrated NPU)
System Memory (RAM)	96.0 GB Configuration (95.5 GB physically addressable software cache)
Graphics Adapter (GPU)	Intel(R) Arc(TM) 140T GPU (48GB Shared Allocatable VRAM)
Target Software OS	Windows 11 Home / Professional (x64-based Architecture)

3. Data Ingestion & Proxy Architecture

Processing 5.7K panoramic spherical layers directly introduces extreme compute and I/O bottlenecks. To guarantee interactive speeds and fast execution loops, the system implements a strict dual-track proxy pipeline:

- **The Ingestion Proxy Track:** The camera generates a paired low-resolution configuration file alongside the raw master container (e.g., `LRV_202605_11_001.mp4` vs `VID_202605_00_001.insv`). The system automatically isolates this pre-stitched `.lrv` file to perform low-overhead structural vision tracking at lower matrix resolutions ($640 imes 480$ / $1280 imes 720$).
- **The Production Execution Track:** Once the tracking maps and coordinate matrices are generated across the low-res timeline, the indices are mathematically scaled to the high-resolution source video track. The final

execution is performed using the pre-stitched equirectangular outputs exported directly via the high-bitrate Insta360 Studio encoder pipeline (100–200 Mbps H.264/H.265 stream).

4. Core AI Vision Stack & Acceleration Layer

To achieve native performance without dedicated Nvidia CUDA tensors, the software relies on the **Intel OpenVINO Toolkit** to convert and optimize open-source architectures to match the processing capabilities of the Ultra 9 NPU and Arc 140T GPU cores.

4.1 Class-Based Tracking (YOLOv10 / RT-DETR)

The default engine utilizes a pre-trained, OpenVINO-quantized object detector configured to scan for action sport classes. The model scans frame matrices to generate standard boundary coordinates around targets mapped inside the flat equirectangular canvas.

4.2 Custom & Generic Object Tracking (SAM 2 Integration)

For non-standard target configurations—such as tracking a moving bowling ball or distinct safety equipment—the engine integrates a localized implementation of the **Segment Anything Model 2 (SAM 2)**. Given the massive 96GB system RAM ceiling, the application caches temporal attention maps locally, allowing the user to click any target on Frame 0, creating a robust pixel-exact track across highly volatile environments.

> **Model Optimization Specification**
>
> All target neural network weights (PyTorch FP32 models) must pass through OpenVINO's Post-Training Optimization Tool (POT) to convert weight configurations into **INT8 Precision**. This optimization reduces memory footprint and maximizes throughput by running parallel inference paths natively on the dedicated Arc GPU.

5. Kinematic Smooth Filter & Coordinates

Raw bounding boxes generated frame-by-frame by AI object detection models suffer from geometric jitter due to changing boundaries, environmental factors, and spray (snow or water). Passing these raw paths directly to a crop tool causes disorienting, unstable video motion.

To solve this, the engine translates 2D bounding boxes into spherical target coordinates: Yaw (horizontal angle panning) and $Pitch$ (vertical tilt angle). This array is then passed through a mathematical smoothing filter to introduce virtual camera inertia.

A discrete **Kalman Filter** tracks the true motion vector of the subject, stripping away tracking noise and predicting trajectories when the subject is obscured by environmental obstacles (e.g., water spray during rafting or snow powder while skiing):

$$\alpha_k = \alpha_{k|k-1} + K_k(z_k - H\alpha_{k|k-1})$$

Where α_k represents the updated, smooth orientation angle array, K_k denotes the calculated Kalman gain adjusting tracking weights, and z_k represents the raw noisy coordinate input derived from the AI bounding box layer. This creates a fluid, cinematic motion path resembling a physical fluid-head tripod or electronic gimbal.

6. Audio Treatment Pipeline

Action sports footage features high-intensity environmental audio that is easily ruined by wind buffeting or water impact clipping against microphone housings.

- **Low-End Frequency Suppression:** A fast 2nd-order High-Pass Filter (HPF) is implemented digitally to aggressively cut off signals below 150Hz, effectively stripping out structural wind noise.
- **Dynamic Range Preservation:** Instead of executing heavy noise cancellation which degrades raw acoustic clarity, the system hooks an audio limiter/compressor. This keeps the impact sound of breaking rapids or edge-carving on ice while preventing high-amplitude peaks from clipping the audio signal.

7. Final Render Core (Intel QuickSync Video)

The final rendering engine relies entirely on a compiled **FFmpeg** distribution integrated directly into the local workspace. Rather than straining the CPU cores for video compression, the rendering loop utilizes **Intel QuickSync Video (QSV)** parameters.

The pipeline reads the full-resolution stitched master frame, references the smoothed Yaw and $Pitch$ coordinates generated during the proxy pass, and extracts a cropped window in standard consumer formats (16:9 widescreen or 9:16 vertical video).

By invoking the `-hwaccel qsv -c:v h264_qsv` or `hevc_qsv` compiler flags, the system performs real-time parallel hardware encoding, generating the final output `.mp4` files at maximum efficiency.